

lux Connection-Lease Reaping — Fidelity Control (Reconnect Races Reap)

Buggy variant: Connect and Renew never wait for an open departure cascade

September 2026

1 Introduction

This is the fidelity control for the `register_client-vs-reap` hazard `docs/architecture/lease-reap-cascade.md` Section 4 reports, and for the `Connect/Renew` mutual-exclusion fix `docs/connection_lease_reaping.tex` adds to close it. It removes only the `cascadeFor = ∅` guard the fixed spec's `Connect` and `Renew` carry: real `register_client/identify_client` do not take `StoreLock` today, so nothing stops a same-identity reconnect from landing between `_depart_lapsed`'s registry-and-ownership removal and its cascade tail, while that tail – decided against stale information – still runs and wipes out the reconnect's just-installed writer binding. Read the main specification first; this document states only what differs, in Section 5.1 and Section 5.2. The `TimedReapBegin/TimedReapEnd` split itself is *not* the bug and is carried over unchanged: it is the fixed real system's own shape (`StoreLock` held across the whole critical section), and this control's only departure from the fixed spec is that `Connect/Renew` do not yet respect it.

2 Basic Types

$[CONN, IDENT, SCENE]$

$TSTATE ::= tup \mid tdown$

$LSTATE ::= llive \mid llapsed$

$OWNER ::= unowned \mid conn \langle\langle CONN \rangle\rangle$

$FSTATE ::= fclear \mid fset$

Unchanged from the main specification.

3 State

ConnReg

$registered : \mathbb{P} CONN$
 $identity : CONN \leftrightarrow IDENT$
 $transport : CONN \leftrightarrow TSTATE$
 $lease : CONN \leftrightarrow LSTATE$
 $owner : SCENE \rightarrow OWNER$
 $hasSubs : CONN \rightarrow FSTATE$
 $hasWriter : CONN \rightarrow FSTATE$
 $inboxNonEmpty : CONN \rightarrow FSTATE$
 $cascadeFor : \mathbb{P} CONN$

$registered \subseteq \text{dom } identity$
 $\text{dom } identity = \text{dom } transport$
 $\text{dom } transport = \text{dom } lease$
 $\#cascadeFor \leq 1$

Unchanged from the main specification.

4 Initialization

<i>Init</i>
<i>ConnReg'</i>
$registered' = \emptyset$ $identity' = \emptyset$ $transport' = \emptyset$ $lease' = \emptyset$ $owner' = (\lambda s : SCENE \bullet unowned)$ $hasSubs' = (\lambda c : CONN \bullet fclear)$ $hasWriter' = (\lambda c : CONN \bullet fclear)$ $inboxNonEmpty' = (\lambda c : CONN \bullet fclear)$ $cascadeFor' = \emptyset$

Unchanged from the main specification.

5 Operations

5.1 Connect — BUGGY: no guard against an open departure cascade

<i>Connect</i>
$\Delta ConnReg$ $c? : CONN$ $i? : IDENT$
$registered' = registered \cup \{c?\}$ $identity' = identity \oplus \{c? \mapsto i?\}$ $transport' = transport \oplus \{c? \mapsto tup\}$ $lease' = lease \oplus \{c? \mapsto llive\}$ $hasWriter' = hasWriter \oplus \{c? \mapsto fset\}$ $hasSubs' = hasSubs$ $inboxNonEmpty' = inboxNonEmpty$ $cascadeFor' = cascadeFor$ $owner' = owner$

The fixed spec's $cascadeFor = \emptyset$ guard is simply absent – everything else, including the unconditional $hasWriter' = hasWriter \oplus \{c? \mapsto fset\}$ representing **ensure_writer**'s net effect, is unchanged.

5.2 Renew — BUGGY: no guard against an open departure cascade

<i>Renew</i>
$\Delta ConnReg$ $c? : CONN$
$c? \in registered$ $transport\ c? = tup$ $lease' = lease \oplus \{c? \mapsto llive\}$ $registered' = registered$ $identity' = identity$ $transport' = transport$ $hasSubs' = hasSubs$ $hasWriter' = hasWriter$ $inboxNonEmpty' = inboxNonEmpty$ $cascadeFor' = cascadeFor$ $owner' = owner$

Activate

$\Delta ConnReg$
 $c? : CONN$

$c? \in registered$
 $transport\ c? = tup$
 $hasSubs' = hasSubs \oplus \{c? \mapsto fset\}$
 $hasWriter' = hasWriter \oplus \{c? \mapsto fset\}$
 $inboxNonEmpty' = inboxNonEmpty \oplus \{c? \mapsto fset\}$
 $registered' = registered$
 $identity' = identity$
 $transport' = transport$
 $lease' = lease$
 $owner' = owner$
 $cascadeFor' = cascadeFor$

TransportDies

$\Delta ConnReg$
 $c? : CONN$

$c? \in registered$
 $transport\ c? = tup$
 $transport' = transport \oplus \{c? \mapsto tdown\}$
 $registered' = registered$
 $identity' = identity$
 $lease' = lease$
 $owner' = owner$
 $hasSubs' = hasSubs$
 $hasWriter' = hasWriter$
 $inboxNonEmpty' = inboxNonEmpty$
 $cascadeFor' = cascadeFor$

LeaseLapse

$\Delta ConnReg$
 $c? : CONN$

$c? \in registered$
 $lease\ c? = llive$
 $lease' = lease \oplus \{c? \mapsto llapsed\}$
 $registered' = registered$
 $identity' = identity$
 $transport' = transport$
 $owner' = owner$
 $hasSubs' = hasSubs$
 $hasWriter' = hasWriter$
 $inboxNonEmpty' = inboxNonEmpty$
 $cascadeFor' = cascadeFor$

Read

$\Xi ConnReg$

Depart

$\Delta ConnReg$
 $c? : CONN$

$c? \in registered$
 $registered' = registered \setminus \{c?\}$
 $owner' = owner \oplus$
 $(\{s : SCENE \mid owner\ s = conn(c?)\} \times \{unowned\})$
 $hasSubs' = hasSubs \oplus \{c? \mapsto fclear\}$
 $hasWriter' = hasWriter \oplus \{c? \mapsto fclear\}$
 $inboxNonEmpty' = inboxNonEmpty \oplus \{c? \mapsto fclear\}$
 $identity' = identity$
 $transport' = transport$
 $lease' = lease$
 $cascadeFor' = cascadeFor$

TimedReapBegin

$\Delta ConnReg$
 $c? : CONN$

$c? \in registered$
 $lease\ c? = llapsed$
 $cascadeFor = \emptyset$
 $registered' = registered \setminus \{c?\}$
 $owner' = owner \oplus$
 $(\{s : SCENE \mid owner\ s = conn(c?)\} \times \{unowned\})$
 $cascadeFor' = \{c?\}$
 $identity' = identity$
 $transport' = transport$
 $lease' = lease$
 $hasSubs' = hasSubs$
 $hasWriter' = hasWriter$
 $inboxNonEmpty' = inboxNonEmpty$

TimedReapEnd

$\Delta ConnReg$
 $c? : CONN$

$cascadeFor = \{c?\}$
 $cascadeFor' = \emptyset$
 $hasSubs' = hasSubs \oplus \{c? \mapsto fclear\}$
 $hasWriter' = hasWriter \oplus \{c? \mapsto fclear\}$
 $inboxNonEmpty' = inboxNonEmpty \oplus \{c? \mapsto fclear\}$
 $registered' = registered$
 $identity' = identity$
 $transport' = transport$
 $lease' = lease$
 $owner' = owner$

Activate, TransportDies, LeaseLapse, Read, Depart, TimedReapBegin, and TimedReapEnd are all unchanged from the fixed spec – the split itself, and the cascade it exposes, are not what this control tests.

Claim

$\Delta ConnReg$

$c? : CONN$

$s? : SCENE$

$c? \in registered$

$transport\ c? = tup$

$lease' = lease \oplus \{c? \mapsto llive\}$

$registered' = registered \setminus$

$\{c2 : CONN \mid c2 \in registered \wedge c2 \neq c? \wedge lease\ c2 = llapsed\}$

$(owner \oplus (\{s : SCENE \mid \exists c2 : CONN \bullet$

$c2 \in registered \wedge c2 \neq c? \wedge lease\ c2 = llapsed \wedge owner\ s = conn(c2)\}$

$\times \{unowned\}))\ s? = unowned$

$\vee$

$(owner \oplus (\{s : SCENE \mid \exists c2 : CONN \bullet$

$c2 \in registered \wedge c2 \neq c? \wedge lease\ c2 = llapsed \wedge owner\ s = conn(c2)\}$

$\times \{unowned\}))\ s? = conn(c?)$

$owner' = (owner \oplus (\{s : SCENE \mid \exists c2 : CONN \bullet$

$c2 \in registered \wedge c2 \neq c? \wedge lease\ c2 = llapsed \wedge owner\ s = conn(c2)\}$

$\times \{unowned\})) \oplus \{s? \mapsto conn(c?)\}$

$hasSubs' = hasSubs \oplus (\{c2 : CONN \mid c2 \in registered \wedge c2 \neq c? \wedge lease\ c2 = llapsed\} \times \{fclear\})$

$hasWriter' = hasWriter \oplus (\{c2 : CONN \mid c2 \in registered \wedge c2 \neq c? \wedge lease\ c2 = llapsed\} \times \{fclear\})$

$inboxNonEmpty' = inboxNonEmpty \oplus (\{c2 : CONN \mid c2 \in registered \wedge c2 \neq c? \wedge lease\ c2 = llapsed\} \times \{fclear\})$

$identity' = identity$

$transport' = transport$

$cascadeFor' = cascadeFor$

Also unchanged.

6 Verification: the goal that must flip

Against this variant, a live, just-reconnected connection can end up with no writer bound – the trace in `connection_lease_reaping.tex`, Section “Reconnect-races-reap,” reaches it in five steps: `Connect(c1, il); LeaseLapse(c1); TimedReapBegin(c1); Connect(c1, il)` again – buggy, no guard refuses it despite `cascadeFor = {c1}; TimedReapEnd(c1)` – the stale cascade tail lands after the reconnect and wipes out the writer it just installed.

```
# must be FOUND against this variant (NOT found against the fixed spec)
probccli docs/connection_lease_reaping_reconnect_race_buggy.tex \
-model_check -nodead \
-goal "#c.(c:CONN & c : registered & hasWriter(c) = fclear)" \
-p DEFAULT_SETSIZE 3
```

I2/I3/I5/I6/I7 must stay unaffected – `Connect/Renew` never touch `registered`, `owner`, `hasSubs`, or `inboxNonEmpty` any differently from the fixed spec, and this variant’s `TimedReapBegin/TimedReapEnd` are the fixed spec’s own:

```
# must remain NOT found
probccli docs/connection_lease_reaping_reconnect_race_buggy.tex \
-model_check -nodead \
-goal "#s.(#c.(s:SCENE & c:CONN & owner(s) = conn(c) &
      not(c : registered)))" \
-p DEFAULT_SETSIZE 3
```

```
# must remain NOT found
probccli docs/connection_lease_reaping_reconnect_race_buggy.tex \
-model_check -nodead \
-goal "#c.(c:CONN & not(c : registered) & not(c : cascadeFor) &
      (hasSubs(c) = fset or hasWriter(c) = fset or
```

```
        inboxNonEmpty(c) = fset))" \
-p DEFAULT_SETSIZE 3
```

I1 must stay unaffected – the chain that establishes I1's positive outcome does not touch **Connect/Renew** at all:

```
# must remain FOUND
probccli docs/connection_lease_reaping_reconnect_race_buggy.tex \
-model_check -nodead \
-goal "#c.(c:CONN & c : dom(identity) & not(c : registered) &
        transport(c) = tdown)" \
-p DEFAULT_SETSIZE 3
```